VisualConverter

Linode Server — Full Deployment Guide

Laravel 10 · PHP 8.1 · FFmpeg · MySQL 8 · Nginx · Supervisor

Generated: 27 May 2026

Project	VisualConverter
Purpose	API + Video Conversion Engine
Server	Linode VPS — Ubuntu 22.04 LTS
Stack	Nginx · PHP 8.1-FPM · MySQL 8 · Supervisor
Queue	Database driver · conversions queue
Storage	Cloudflare R2 (private + public buckets)

Table of Contents

Phase 1 — Initial Server Hardening

Run these commands immediately after first SSH login as root.

```
# Update all packages
apt update && apt upgrade -y

# Set a recognisable hostname
hostnamectl set-hostname visualconverter

# Set timezone
timedatectl set-timezone Asia/Kolkata

# Create a non-root deploy user
adduser deploy
usermod -aG sudo deploy

# Copy your SSH public key to the deploy user
rsync --archive --chown=deploy:deploy ~/.ssh /home/deploy

# Firewall — allow SSH, HTTP, HTTPS only
ufw allow OpenSSH
ufw allow 80
ufw allow 443
ufw enable
ufw status
```

■ *After this step, log out and log back in as the deploy user for all future steps.*

Phase 2 — Install Nginx

```
apt install -y nginx

systemctl start nginx
systemctl enable nginx

# Verify installation
nginx -v
```

Phase 3 — Install PHP 8.1 + Extensions

Add the Ondrej PPA to get PHP 8.1 on Ubuntu 22.04.

```
# Add PHP repository
apt install -y software-properties-common
add-apt-repository ppa:ondrej/php -y
apt update

# Install PHP 8.1 + all required extensions
apt install -y \
  php8.1-fpm     \
  php8.1-cli     \
  php8.1-mysql   \
  php8.1-mbstring \
  php8.1-xml     \
  php8.1-curl    \
  php8.1-zip     \
  php8.1-bcmath \
  php8.1-gd      \
  php8.1-intl    \
  php8.1-fileinfo \
  php8.1-tokenizer

# Verify
php -v

# Start PHP-FPM
systemctl start php8.1-fpm
systemctl enable php8.1-fpm
```

Phase 4 — Install MySQL 8.0

```
apt install -y mysql-server

systemctl start mysql
systemctl enable mysql

# Interactive security hardening
mysql_secure_installation
# → Set root password
# → Remove anonymous users     : Y
# → Disallow root login remotely: Y
# → Remove test database        : Y
# → Reload privilege tables     : Y
```

Then create the project database and dedicated user:

```
mysql -u root -p
```

```
-- Run inside MySQL shell
CREATE DATABASE visualdepotconverter CHARACTER SET utf8mb4 COLLATE utf8mb4_unicode_ci;
CREATE USER 'visualconv'@'localhost' IDENTIFIED BY 'StrongPassword@123';
GRANT ALL PRIVILEGES ON visualdepotconverter.* TO 'visualconv'@'localhost';
FLUSH PRIVILEGES;
EXIT;
```

Phase 5 — Install Composer & Git

```
# Git
apt install -y git

# Composer
curl -sS https://getcomposer.org/installer | php
mv composer.phar /usr/local/bin/composer
chmod +x /usr/local/bin/composer

# Verify
composer --version
git --version
```

Phase 6 — Install FFmpeg

FFmpeg is the core of the conversion engine. Verify that libx264 and aac codecs are present.

```
apt install -y ffmpeg

# Verify version
ffmpeg -version

# Confirm required codecs are available
ffmpeg -codecs 2>/dev/null | grep -E "libx264|aac"

# Note the binary paths — needed for .env
which ffmpeg    # → /usr/bin/ffmpeg
which ffprobe   # → /usr/bin/ffprobe
```

■■ If libx264 is missing, run: apt install -y libx264-dev

Phase 7 — Install Supervisor (Queue Worker Daemon)

```
apt install -y supervisor

systemctl start supervisor
systemctl enable supervisor
```

Phase 8 — Deploy the Project

Switch to the deploy user and pull the project onto the server.

```
# Switch to deploy user
su - deploy

# Create web root
sudo mkdir -p /var/www/visualconverter
sudo chown deploy:deploy /var/www/visualconverter

# Option A — clone from Git remote
git clone https://github.com/YOUR_USERNAME/visualconverter.git /var/www/visualconverter

# Option B — push from local machine using rsync
# rsync -avz --exclude='.git' --exclude='node_modules' --exclude='vendor' \
#   /Users/jitendra/project/mithilesh/visualconverter/ \
#   deploy@YOUR_SERVER_IP:/var/www/visualconverter/

# Move into the project
cd /var/www/visualconverter

# Install PHP dependencies (no dev packages on production)
composer install --no-dev --optimize-autoloader
```

Phase 9 — Configure .env

```
cp .env.example .env
nano .env
```

Fill in the following values:

```
APP_NAME=VisualConverter
APP_ENV=production
APP_KEY=                          # generated below
APP_DEBUG=false
APP_URL=https://YOUR_DOMAIN_OR_IP

# Shared secret — main site sends this as Bearer token
API_TOKEN=GENERATE_A_LONG_RANDOM_STRING_HERE

# Callback to main site after each conversion completes / fails
CALLBACK_URL=https://yourmainsite.com/api/video-converted
CALLBACK_TOKEN=your-main-site-callback-token

LOG_CHANNEL=stack
LOG_LEVEL=warning

DB_CONNECTION=mysql
DB_HOST=127.0.0.1
DB_PORT=3306
DB_DATABASE=visualdepotconverter
DB_USERNAME=visualconv
DB_PASSWORD=StrongPassword@123

QUEUE_CONNECTION=database

# Cloudflare R2 — private bucket (source + converted videos)
R2_ACCESS_KEY_ID=5f151858d59b4c20173929dda4c4e5fd
R2_SECRET_ACCESS_KEY=1197ec611f3f134d60c25e1d7f77a0e0b4152affc572813abd2994d0ec58df3b
R2_BUCKET=visualsdepot-private
R2_ENDPOINT=https://4ede5328b8f6775193d38e85306270a7.r2.cloudflarestorage.com

# Cloudflare R2 — public bucket (watermarked videos)
R2_PUBLIC_ACCESS_KEY_ID=5f151858d59b4c20173929dda4c4e5fd
R2_PUBLIC_SECRET_ACCESS_KEY=1197ec611f3f134d60c25e1d7f77a0e0b4152affc572813abd2994d0ec58df3b
R2_PUBLIC_BUCKET=visualsdepot-public
R2_PUBLIC_URL=https://pub-b40c47db69e74b119472e30b1f9a5878.r2.dev

# FFmpeg binary paths
FFMPEG_BINARIES=/usr/bin/ffmpeg
FFPROBE_BINARIES=/usr/bin/ffprobe
```

```
# Generate app key
php artisan key:generate

# Run database migrations
php artisan migrate --force

# Cache config + routes for production performance
php artisan config:cache
php artisan route:cache

# Create temp directory for FFmpeg processing
mkdir -p storage/app/temp
```

Phase 10 — File Permissions

```
cd /var/www/visualconverter

sudo chown -R deploy:www-data .
sudo chmod -R 755 .
sudo chmod -R 775 storage bootstrap/cache
```

Phase 11 — Nginx Virtual Host

```
sudo nano /etc/nginx/sites-available/visualconverter
```

Paste the following configuration:

```
server {
    listen 80;
    server_name YOUR_DOMAIN_OR_IP;

    root /var/www/visualconverter/public;
    index index.php;

    client_max_body_size 10M;
    fastcgi_read_timeout 120;
    proxy_read_timeout 120;

    location / {
        try_files $uri $uri/ /index.php?$query_string;
    }

    location ~ \.php$ {
        include snippets/fastcgi-php.conf;
        fastcgi_pass unix:/run/php/php8.1-fpm.sock;
        fastcgi_param SCRIPT_FILENAME $realpath_root$fastcgi_script_name;
    }

    location ~ /\.(?!well-known).* {
        deny all;
    }

    location ~* \.(env|log|htaccess)$ {
        deny all;
    }

    access_log /var/log/nginx/visualconverter_access.log;
    error_log  /var/log/nginx/visualconverter_error.log;
}
```

```
# Enable the site
sudo ln -s /etc/nginx/sites-available/visualconverter /etc/nginx/sites-enabled/

# Remove default placeholder site
sudo rm -f /etc/nginx/sites-enabled/default

# Test configuration syntax
sudo nginx -t

# Apply changes
sudo systemctl reload nginx
```

Phase 12 — Supervisor — Queue Worker

```
sudo nano /etc/supervisor/conf.d/visualconverter-worker.conf
```

Paste the following configuration:

```
[program:visualconverter-worker]
process_name=%(program_name)s_%(process_num)02d
command=php /var/www/visualconverter/artisan queue:work database \
        --queue=conversions \
        --sleep=3 \
        --tries=2 \
        --timeout=7200 \
        --max-jobs=50 \
        --max-time=3600
autostart=true
autorestart=true
stopasgroup=true
killasgroup=true
user=deploy
numprocs=1
redirect_stderr=true
stdout_logfile=/var/www/visualconverter/storage/logs/worker.log
stdout_logfile_maxbytes=50MB
stdout_logfile_backups=5
stopwaitsecs=7300
```

■ *numprocs=1 keeps one worker at a time. Videos are CPU-heavy — do not run parallel conversions unless the server has 4+ cores and 8 GB+ RAM.*

```
# Load and start the worker
sudo supervisorctl reread
sudo supervisorctl update
sudo supervisorctl start visualconverter-worker:*

# Verify it is running
sudo supervisorctl status
```

Phase 13 — SSL with Let's Encrypt (Domain Required)

```
apt install -y certbot python3-certbot-nginx

certbot --nginx -d yourdomain.com

# Test automatic renewal
certbot renew --dry-run
```

■ *Certbot automatically updates the Nginx config to redirect HTTP to HTTPS and renews certificates every 90 days via a cron job.*

Phase 14 — Verify Everything Works

```
# 1. Check all services are running
systemctl status php8.1-fpm
systemctl status nginx
systemctl status mysql
sudo supervisorctl status

# 2. Test the API endpoint
curl -X POST http://YOUR_DOMAIN_OR_IP/api/v1/convert \
  -H "Authorization: Bearer YOUR_API_TOKEN" \
  -H "Content-Type: application/json" \
  -d '{
    "r2_path": "originals/sample.mp4",
    "source_quality": "4K",
    "destination_quality": "HD",
    "destination_folder": "converted/",
    "external_video_id": "1"
  }'

# Expected response (202 Accepted):
# { "message": "Conversion jobs queued.", "jobs": [...] }

# 3. Watch the queue worker log live
tail -f /var/www/visualconverter/storage/logs/worker.log

# 4. Watch Laravel application log
tail -f /var/www/visualconverter/storage/logs/laravel.log
```

Quick Reference — Useful Commands

```
# ■■ After every code deployment ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
cd /var/www/visualconverter
composer install --no-dev --optimize-autoloader
php artisan migrate --force
php artisan config:cache
php artisan route:cache
sudo supervisorctl restart visualconverter-worker:*

# ■■ Config changes only (.env edited) ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
php artisan config:clear && php artisan config:cache
sudo supervisorctl restart visualconverter-worker:*

# ■■ Queue management ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
# List failed jobs
php artisan queue:failed

# Retry a specific failed job
php artisan queue:retry JOB_ID

# Retry all failed jobs
php artisan queue:retry all

# Clear all failed jobs
php artisan queue:flush

# ■■ Supervisor management ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
sudo supervisorctl status
sudo supervisorctl restart visualconverter-worker:*
sudo supervisorctl stop visualconverter-worker:*
sudo supervisorctl start visualconverter-worker:*

# ■■ Log monitoring ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
tail -f /var/www/visualconverter/storage/logs/laravel.log
tail -f /var/www/visualconverter/storage/logs/worker.log
tail -f /var/log/nginx/visualconverter_error.log
```

Linode Recommended Plan

Resource	Minimum	Recommended
CPU	2 cores	4+ cores
RAM	4 GB	8 GB
Disk	80 GB SSD	160 GB SSD
Network	1 Gbps	1 Gbps

■■ Disk space is critical. A single 8K source file can be 10–50 GB. During conversion you will have the source + 4K output + HD output on disk simultaneously. Always monitor free space with: df -h